

PROCESOS DE DESARROLLO II

Diseño de software, arquitectura y patrones de diseño

Clase 1 — Presentación de la materia

Agenda

Qué vamos a recorrer en esta primera clase



Programa de la materia

Objetivos, ejes temáticos y modalidad de cursada



Bibliografía

Fuentes clásicas y complementarias del curso



Aula virtual

Dónde vas a encontrar material, entregas y avisos



Clases y actividades

Formato de encuentros y trabajo práctico continuo



Proyecto integrador

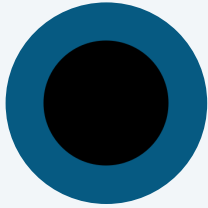
El trabajo que vertebra toda la cursada



Repaso de conceptos

Lo que traemos de Procesos de desarrollo I

Objetivos de la materia



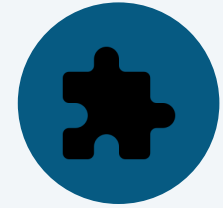
Diseñar con calidad

Identificar las áreas de la etapa de diseño y su relevancia en el proceso de desarrollo.



Elegir la arquitectura correcta

Reconocer qué modelo arquitectónico corresponde según la naturaleza del software a construir.



Aplicar patrones de diseño

Usarlos como herramienta concreta para mejorar la calidad del software que construimos.

Temas relevantes

El recorrido conceptual de la cursada



Diseños de calidad

Cómo reconocer y producir un buen diseño de software



Buenas prácticas de IS

Principios que sostienen el trabajo de ingeniería



Mejorando el modelado

UML y diagramas como herramienta de comunicación



Proyecto con metodologías ágiles

Desarrollo iterativo aplicado a un caso real



Patrones de diseño

Soluciones reutilizables a problemas recurrentes

Bibliografía



Referencias centrales

- Pressman, R. — Ingeniería de software: un enfoque práctico (7.^a ed.)
- Gamma, Helm, Johnson, Vlissides — Patrones de diseño
- Larman, C. — UML y patrones
- Jacobson, Booch, Rumbaugh — El lenguaje unificado de modelado UML 2.0

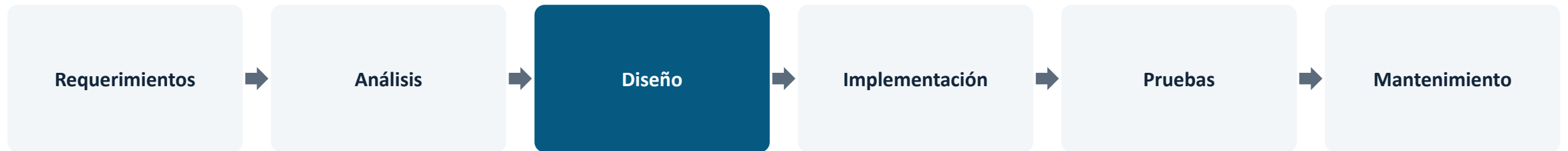


Complementarias

- Arlow, J. & Neustadt, I. — Programación UML 2
- Weitzenfeld, A. — Ingeniería de software orientada a objetos con UML, Java e Internet
- Priolo, S. — Metodologías ágiles
- Brooks, F. — The Design of Design (2010)

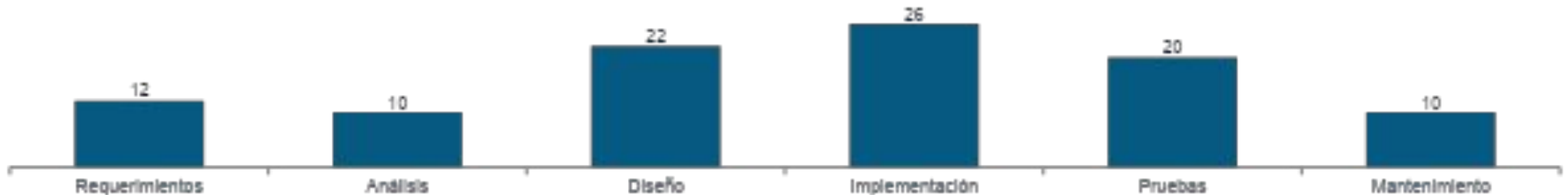
Repaso: el ciclo de vida del desarrollo

Las etapas que venimos trabajando desde Procesos de desarrollo I



Esta materia profundiza especialmente la etapa de diseño: modelos arquitectónicos y patrones.

Distribución habitual del esfuerzo por etapa



Niveles de abstracción en el diseño

Cada nivel responde una pregunta distinta sobre el sistema

Arquitectura

¿Cómo se organizan los grandes bloques del sistema?

Subsistemas / módulos

¿Qué responsabilidad tiene cada parte?

Clases de diseño

¿Qué objetos concretos implementan cada módulo?

Código

¿Cómo se traduce cada clase a un lenguaje concreto?



Bajar de nivel implica ganar detalle y perder visión de conjunto — por eso se documentan con diagramas distintos.

Atributos de calidad del software

Con ejemplos de la industria actual



Modularidad

Netflix y Spotify dividen su backend en microservicios independientes.



Usabilidad

Apps mobile-first como MercadoPago priorizan flujos simples de pago.



Confiabilidad

Sistemas de pago bancarios operan con objetivos de 99,99% de disponibilidad.



Robustez

El software de aviónica debe seguir operando ante datos de sensores inválidos.

Atributos de calidad del software (cont.)



Complejidad y arquitectura

El kernel de Linux gestiona millones de líneas mediante arquitectura en capas.



Reusabilidad

Paquetes de npm y componentes de React se reutilizan en miles de proyectos.



Portabilidad

Flutter y React Native compilan una sola base de código a iOS y Android.



Mantenibilidad

Pipelines de CI/CD y testing automatizado sostienen cambios frecuentes y seguros.

Para repasar juntos

Preguntas disparadoras antes de avanzar

1

¿Cuáles son los atributos de calidad de un buen software?

2

¿Qué representa un diagrama de actividad?

3

¿Qué son los niveles de abstracción y para qué sirven?

4

¿Cómo validamos que un diagrama de clases está bien estructurado?

5

¿Qué dificultades existen al relevar requerimientos?

6

¿Qué hacemos concretamente cuando diseñamos software?